The Reward Model & Bradley–Terry.

How do we turn a pile of *human preferences* — "answer A is better than answer B" — into a differentiable scalar reward? This topic builds the reward model $r_\phi(x, y)$ from the Bradley–Terry model of pairwise choice, derives its loss and gradient, and computes every number by hand.

THE EQUATION

$$\mathcal{L}_{\text{RM}}(\phi) \;=\; -\,\mathbb{E}_{(x,y_w,y_l)}\Big[\log \sigma\big(r_\phi(x,y_w) - r_\phi(x,y_l)\big)\Big]$$

Worked example. Four preference pairs with concrete reward scores; we compute each reward gap, its sigmoid (the model's preference probability), the per-pair loss, and the mean.

Read top to bottom: each step feeds the next.

Contents

1 The three-step recipe of InstructGPT

RLHF (Reinforcement Learning from Human Feedback) aligns a pretrained language model with human intent in three stages. This module follows the InstructGPT pipeline:

1. **SFT — supervised fine-tuning.** Fine-tune the base model on human demonstrations $(x, y^\star)$ by plain next-token negative log-likelihood. This gives $\pi_{\text{ref}} = \pi_{\text{SFT}}$.
2. **RM — reward modelling.** Collect *preference* pairs (x, y_w, y_l), where a human judged the "winner" y_w better than the "loser" y_l, and fit a scalar reward $r_\phi(x, y)$. **(This topic.)**
3. **RL — policy optimization.** Optimize the policy against r_ϕ with a KL leash to π_{ref} (Topic 2), or skip the reward model entirely with DPO (Topic 3).

1.1 Step 1 in equations: the SFT loss

Given a demonstration $y^\star = (y_1^\star, \dots, y_T^\star)$ for prompt x, supervised fine-tuning is ordinary autoregressive maximum likelihood — it has *nothing* reinforcement-specific about it:

$$\boxed{\mathcal{L}_{\text{SFT}}(\theta) \;=\; -\,\mathbb{E}_{(x,y^\star)} \sum_{t=1}^{T} \log \pi_\theta(y_t^\star \mid x, y_{<t}^\star)} \tag{1}$$

This is the same cross-entropy used in pretraining; it produces a model that *imitates* demonstrations. The remaining steps go beyond imitation toward *preference*.

SFT teaches the model *what a good answer looks like*; the reward model teaches it *how to compare two answers*. Humans are far more reliable at ranking (A vs. B) than at scoring on an absolute scale, so we collect comparisons and let the model infer an underlying scalar.

2 Bradley–Terry: from comparisons to a scalar

The Bradley–Terry (1952) model posits a latent "strength" s_i for each item, with

$$P(i \succ j) \;=\; \frac{e^{s_i}}{e^{s_i} + e^{s_j}}.$$

We identify the latent strength with the reward, $s_i = r_\phi(x, y_i)$. Dividing numerator and denominator by e^{s_i}:

$$P(y_w \succ y_l) = \frac{e^{r_w}}{e^{r_w} + e^{r_l}} = \frac{1}{1 + e^{-(r_w - r_l)}} = \sigma\big(r_w - r_l\big),$$

where $r_w \equiv r_\phi(x, y_w)$, $r_l \equiv r_\phi(x, y_l)$, and $\sigma(z) = 1/(1 + e^{-z})$ is the logistic sigmoid.

$$\boxed{P(y_w \succ y_l \mid x) \;=\; \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)} \tag{2}$$

Only the *difference* of rewards matters: r_ϕ is identified up to an additive constant per prompt. The reward model is a copy of the transformer with the LM head replaced by a single scalar output, read off the final token's hidden state.

Symbol	Meaning	Shape
x	prompt	token sequence
y_w, y_l	winning / losing response	token sequences
$r_\phi(x, y)$	scalar reward (RM output)	$\mathbb{R}$
$\sigma(z)$	logistic sigmoid $1/(1 + e^{-z})$	$\mathbb{R} \to (0, 1)$
$\mathcal{L}_{\text{RM}}$	negative log-likelihood of preferences	$\mathbb{R}_{\geq 0}$

3 The reward-model loss and its gradient

Maximum likelihood over the preference dataset means minimising the negative log-likelihood of the Bradley–Terry probabilities:

$$\boxed{\mathcal{L}_{\text{RM}}(\phi) = -\,\mathbb{E}_{(x, y_w, y_l)}\Big[\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)\Big]} \tag{3}$$

3.1 Gradient w.r.t. the reward gap

Write the per-example gap $g = r_w - r_l$ and loss $\ell = -\log \sigma(g)$. Using the identities $\frac{d}{dg} \log \sigma(g) = 1 - \sigma(g) = \sigma(-g)$,

$$\frac{\partial \ell}{\partial g} \;=\; -\big(1 - \sigma(g)\big) \;=\; -\,\sigma(-g) \;<\; 0.$$

Because the derivative is strictly negative, gradient descent *increases* the gap g: it pushes r_w up and r_l down. Chaining through the parameters,

$$\nabla_\phi \ell \;=\; -\,(1-\sigma(g))\,(\nabla_\phi r_w - \nabla_\phi r_l).$$

The scalar prefactor $1-\sigma(g)$ is the *error*: it is large (≈ 1) when the model is wrong or unsure ($g \le 0$) and small (≈ 0) when the model is already confidently correct ($g \gg 0$). This is exactly logistic-regression behaviour: hard, misranked pairs dominate the update.

> The reward-model loss is binary logistic regression on the *difference* of two scalar heads. Its gradient is proportional to $1-\sigma(\text{gap})$, so it relentlessly widens the reward gap between preferred and dispreferred answers until they are confidently separated.

4 Worked computation

Take four preference pairs. Each row lists the reward the model currently assigns to the winner and loser. We compute the gap $g = r_w - r_l$, the preference probability $\sigma(g)$ from Eq. (2), and the per-pair loss $\ell = -\log\sigma(g)$ from Eq. (3).

Pair	r_w	r_l	$g = r_w - r_l$	$\sigma(g)$	$\ell = -\log\sigma(g)$
1	2.1	0.8	1.3	0.785835	0.241008
2	1.5	1.2	0.3	0.574443	0.554355
3	0.4	−0.6	1.0	0.731059	0.313262
4	3.0	2.7	0.3	0.574443	0.554355
			mean loss $\bar{\mathcal{L}}_{\text{RM}} =$		**0.415745**

Step-by-step for pair 1: $g = 2.1 - 0.8 = 1.3$; $\sigma(1.3) = \dfrac{1}{1+e^{-1.3}} = \dfrac{1}{1+0.272532} = 0.785835$; $\ell = -\log(0.785835) = 0.241008$.

Notice pairs 2 and 4 share the same gap (0.3) and hence the same loss (0.554355), even though their *absolute* rewards differ (1.5/1.2 vs. 3.0/2.7) — confirming the additive-constant invariance of Eq. (2).

4.1 Per-pair gradient signal

The prefactor $-(1-\sigma(g))$ tells us how hard each pair pulls:

$$\text{pair 1: } -0.214165, \quad \text{pair 2: } -0.425557, \quad \text{pair 3: } -0.268941, \quad \text{pair 4: } -0.425557.$$

Pairs 2 and 4 (smallest gap) generate the strongest gradient; the well-separated pair 1 contributes least, as expected.

5 Heatmap: pairwise win probabilities

Consider four candidate responses with rewards $r = (2.1,\ 1.5,\ 0.4,\ 3.0)$, indexed $R_1, \ldots, R_4$. Entry (i, j) is $\sigma(r_i - r_j)$, the modelled probability that R_i beats R_j. The matrix is antisymmetric about 0.5 (since $\sigma(z) + \sigma(-z) = 1$) with 0.5 on the diagonal.

	R_1	R_2	R_3	R_4
R_1	0.5000	0.6457	**0.8455**	0.2891
R_2	0.3543	0.5000	0.7503	0.1824
R_3	0.1545	0.2497	0.5000	0.0691
R_4	0.7109	0.8176	**0.9309**	0.5000

R_4 (highest reward) dominates every column; R_3 (lowest) loses every comparison. The reward model has induced a *total order* consistent with the rewards, from purely pairwise data.

A reward model only learns differences, so its absolute scale is arbitrary and may drift during training. Worse, it is trained on the SFT model's outputs — it is reliable only *near* that distribution. Optimising a policy too far against a fixed r_ϕ leads to **reward hacking**: the policy finds inputs where r_ϕ is wrongly high. Topic 2's KL penalty exists precisely to prevent this.

6 Properties that matter

1. **Shift invariance.** $r_\phi \mapsto r_\phi + c(x)$ leaves Eqs. (2)–(3) unchanged.
2. **Convex in the gap.** $\ell(g) = \log(1 + e^{-g})$ is convex and monotonically decreasing; the global pull is always toward larger gaps.
3. **Calibrated probabilities.** The fitted $\sigma(\cdot)$ outputs valid win-probabilities, usable for ranking, rejection sampling, or best-of-n selection.
4. **Reduces to logistic regression** when r_ϕ is linear in features.

7 Where this sits in the track

Step	Object trained	Topic
1. SFT	policy π_{ref} via NLL	here (Eq. 1)
2. RM	reward r_ϕ via Bradley–Terry	**this topic**
3a. PPO	policy with KL leash on r_ϕ	Topic 2
3b. DPO	policy directly, no RM	Topic 3

A Reproducible (NumPy)

Inputs: pairs $(r_w, r_l) \in \{(2.1, 0.8), (1.5, 1.2), (0.4, -0.6), (3.0, 2.7)\}$; four-response rewards $r = (2.1, 1.5, 0.4, 3.0)$. Outputs match the tables above; mean loss $= 0.415745$.

```
import numpy as np
sig = lambda z: 1/(1+np.exp(-z))

pairs = [(2.1,0.8),(1.5,1.2),(0.4,-0.6),(3.0,2.7)]
losses = []
```

```
for rw, rl in pairs:
    g = rw - rl
    p = sig(g)
    losses.append(-np.log(p))
    # gradient prefactor on the gap: -(1 - sigma(g))
print("per-pair loss :", np.round(losses, 6))
print("mean loss     :", np.mean(losses))   # 0.4157451575720673

r = np.array([2.1, 1.5, 0.4, 3.0])
M = sig(r[:, None] - r[None, :])             # pairwise win-prob matrix
print(np.round(M, 4))
# [[0.5    0.6457 0.8455 0.2891]
#  [0.3543 0.5    0.7503 0.1824]
#  [0.1545 0.2497 0.5    0.0691]
#  [0.7109 0.8176 0.9309 0.5   ]]
```